

UNIVERSIDAD NACIONAL DEL ALTIPLANO

Puno

Escuela Profesional de Ingeniería de Sistemas

Práctica 01 — Búsquedas y Notación Asintótica

Implementación y comparación experimental en Python y C++

Repositorio GitHub:

`github.com/handdycurasi-crypto/SIS210-Practica0-Busquedas`

Curso

Algoritmos y Estructuras de Datos – SIS210

Docente

ZANABRIA GALVEZ ALDO HERNAN

Integrantes

YANA MENDOZA CARLOS BENEDICTO

BELIZARIO YANA DAVID VICTOR

CURASI ZEVALLOS HANDDY RONALD

Puno – Perú

2026

Índice

1	Estado del arte y marco teórico ampliado	2
1.1	Núcleo 1. Problema de búsqueda, contrato y representación	2
1.2	Núcleo 2. Modelo RAM, operaciones y costo espacial	2
1.3	Núcleo 3. Significado y uso correcto de O , Ω y Θ	2
1.4	Núcleo 4. Mejor caso, promedio, peor caso y mezcla de consultas	3
1.5	Núcleo 5. Correctitud, invariantes y terminación	3
1.6	Núcleo 6. Búsqueda lineal como referencia y solución práctica	3
1.7	Núcleo 7. Búsqueda binaria: reducción, recurrencia y garantía	4
1.8	Núcleo 8. Búsqueda exponencial y límite efectivo desconocido	4
1.9	Núcleo 9. Interpolación y modelo de posición por valor	4
1.10	Núcleo 10. Distribución, duplicados y selección de hits y misses	5
1.11	Núcleo 11. Rendimiento real, instrumentación y reproducibilidad	5
1.12	Núcleo 12. Selección del algoritmo y amortización del ordenamiento	5
2	Metodología y correctitud	6
3	Entorno experimental y límites del diseño	6
4	Resultados experimentales	7
5	Discusión: respuestas a las preguntas de la práctica	8
6	Heurística propuesta	8
7	Conclusiones	8
	Referencias	9

1. Estado del arte y marco teórico ampliado

1.1 Núcleo 1. Problema de búsqueda, contrato y representación

Una búsqueda debe definirse antes de medirla: dada una secuencia a de longitud n y una clave x , se pide devolver un índice válido que cumpla $a[i] = x$, o informar ausencia. El contrato distingue la operación abstracta de su implementación y evita confundir pertenencia con recuperación de la primera aparición. Morin (s. f., sección 1.2) desarrolla esta separación mediante interfaces; Sedgewick y Wayne (2011, sección 1.1) presentan la búsqueda sobre arreglos como una operación concreta; Erickson (2019, capítulo 0) exige especificaciones precisas para argumentar corrección. En esta práctica, cuando existen duplicados, cualquier coincidencia satisface el contrato: dos índices distintos pueden ser igualmente correctos.

La representación condiciona qué estrategias son aplicables. Un arreglo ofrece posiciones indexadas; una fuente secuencial obliga a consumir elementos anteriores para alcanzar los siguientes. Morin (s. f., sección 1.4) explicita el modelo de acceso, mientras Sedgewick y Wayne (2011) conectan la representación con las operaciones del programa. La perspectiva de diseño de Skiena (2020) ayuda a formular la decisión práctica: primero reconocer las restricciones y después elegir la técnica. Aplicado aquí, el certificado de orden es información útil, pero su ausencia no puede reemplazarse por una suposición. Buscar correctamente en una lista desordenada y buscar rápidamente en una lista ya ordenada son contratos de entrada diferentes, aunque ambos devuelvan un índice.

Aplicación razonada. Para $a = [4, 4, 9]$ y $x = 4$, los índices 0 y 1 son válidos; para $x = 7$ solo lo es -1 . La comprobación usada por el proyecto refleja exactamente esta condición. No exige índices idénticos entre algoritmos como requisito general de correctitud.

1.2 Núcleo 2. Modelo RAM, operaciones y costo espacial

El modelo RAM permite estudiar el número de operaciones sin fijar primero una velocidad de procesador. Se supone acceso a una palabra de memoria y operaciones elementales con costo acotado. Morin (s. f., sección 1.4) precisa el modelo *word-RAM*; el enfoque de Cormen et al. (2022) proporciona el marco del análisis de algoritmos, y Sedgewick y Wayne (2011, sección 1.4) distinguen frecuencia y costo de las instrucciones. Esta abstracción hace comparable el trabajo de dos procedimientos, pero no afirma que una división, una comparación y un acceso real a memoria duren lo mismo. El conteo necesita declarar qué considera operación y qué información deja fuera.

La memoria auxiliar también debe separarse del tamaño de entrada. Las cuatro versiones iterativas conservan un número constante de índices y variables, por lo que utilizan $O(1)$ palabras auxiliares; almacenar el arreglo y las consultas exige memoria adicional. Morin (s. f., sección 1.5) diferencia recursos de representación y operación; Erickson (2019) advierte que el análisis depende del modelo, y la documentación de Python describe enteros con precisión no limitada a una palabra fija (Python Software Foundation, s. f.-a). Por ello, el supuesto de aritmética constante se aplica aquí a las magnitudes acotadas del experimento: no debe extrapolarse a enteros arbitrariamente grandes. En C++, además, se amplía el cálculo antes de restar y multiplicar para evitar desbordamientos en la interpolación de estas entradas.

Aplicación razonada. Guardar ℓ , h y m no multiplica la memoria auxiliar por n . En cambio, copiar el arreglo para ordenarlo sí introduce almacenamiento proporcional a n . El cronómetro del proyecto excluye esa preparación, de modo que sus tiempos no son el costo de una solución completa que primero deba ordenar.

1.3 Núcleo 3. Significado y uso correcto de O , Ω y Θ

La notación asintótica describe cotas de crecimiento, no unidades de tiempo. Para funciones no negativas, $T(n) \in O(g(n))$ si existen $c > 0$ y n_0 tales que $T(n) \leq cg(n)$ para todo $n \geq n_0$; Ω invierte la desigualdad y Θ exige ambas cotas. El tratamiento de Cormen et al. (2022), el trasfondo matemático de Morin (s. f., sección 1.3) y el análisis de recurrencias de Erickson (2019) son complementarios: una cota solo tiene sentido cuando se define la función analizada. Escribir $O(n)$ no significa que se ejecuten exactamente n comparaciones ni que el tiempo se duplique con precisión al duplicar una entrada concreta.

Para comparar búsquedas interesa una cota ajustada cuando pueda justificarse. Por ejemplo, $3n + 7$ pertenece a $\Theta(n)$ y también a $O(n^2)$, pero la segunda afirmación comunica menos. La distinción entre cota válida y descripción informativa se apoya en Morin (s. f.), Cormen et al. (2022) y Erickson (2019); la aplicación algebraica siguiente es una elaboración propia. Para $n \geq 1$, $3n \leq 3n + 7 \leq 10n$, por lo que existen las dos cotas lineales. Asimismo, cambiar la base de $\log n$ solo multiplica por una constante. Sin embargo, $\log \log n$ no puede reemplazarse por una constante en un análisis general, aunque cambie muy lentamente en los tres tamaños medidos.

Consecuencia para el informe. Las mediciones finitas pueden ser compatibles con una función de crecimiento; no establecen por sí mismas una desigualdad válida para todos los tamaños futuros. Una demostración asintótica necesita un argumento sobre el algoritmo, no solo puntos en un gráfico.

1.4 Núcleo 4. Mejor caso, promedio, peor caso y mezcla de consultas

Los casos y las notaciones responden preguntas diferentes. El mejor caso minimiza el costo entre entradas de tamaño n ; el peor lo maximiza; el promedio pondera los costos mediante una distribución explícita. Morin (s.f., sección 1.5) distingue garantías y expectativas; Erickson (2019) explica por qué un argumento debe cubrir las entradas admitidas, y Sedgewick y Wayne (2011, sección 1.4) señalan la dependencia del rendimiento respecto de la entrada. Por tanto, Ω no es un sinónimo de mejor caso ni Θ de promedio. Una búsqueda binaria puede tener mejor caso $\Theta(1)$ y peor caso $\Theta(\log n)$ sin que exista contradicción.

La mezcla 70/30 de esta práctica define una carga concreta: 70% de aciertos y 30% de ausencias. El promedio observado depende también de qué posiciones y qué claves ausentes se seleccionan. La necesidad de explicitar distribuciones se relaciona con Sedgewick y Wayne (2011), el análisis probabilístico de Morin (s.f., sección 1.3) y la dependencia de la distribución estudiada por Van Sandt et al. (2019). El modelo propio para lineal, con claves únicas y hits equiprobables por posición, produce

$$E[C] = 0.7 \frac{n+1}{2} + 0.3n = 0.65n + 0.35.$$

No es un nuevo resultado experimental: es una expectativa teórica con la cual interpretar el conteo existente. Los duplicados y el retorno de la primera coincidencia pueden modificarla.

Lectura crítica. Un miss por encima del máximo puede descartarse inmediatamente en interpolación, mientras lineal recorre todo el arreglo. Cambiar la ubicación de los misses manteniendo el 30% puede cambiar el tiempo promedio. La proporción por sí sola no describe completamente el experimento.

1.5 Núcleo 5. Correctitud, invariantes y terminación

Una implementación eficiente debe primero satisfacer su contrato. La correctitud parcial establece que la respuesta es válida si termina; la terminación completa el argumento. Erickson (2019, capítulo 0) presenta el razonamiento de corrección, Morin (s.f., sección 1.5) lo distingue del costo y Sedgewick y Wayne (2011, sección 1.1) ofrecen la búsqueda binaria como programa verificable. En las búsquedas por intervalo, un invariante apropiado es: si la clave está presente y aún no fue encontrada, permanece entre los límites activos. Esta formulación permite explicar cada descarte en vez de confiar en que un conjunto de ejemplos haya funcionado.

Para binaria, el orden permite conservar el invariante al descartar la mitad incompatible con la comparación central; actualizar a $m+1$ o $m-1$ hace disminuir el intervalo. Para interpolación también debe garantizarse avance, aunque su estimación no divida por la mitad. Esta aplicación combina el esquema de prueba de Erickson (2019), la especificación de algoritmos de Sedgewick y Wayne (2011) y la estimación por valores de Perl et al. (1978). En el proyecto, el vacío se controla antes de indexar y la igualdad de extremos antes de dividir. Las pruebas existentes aportan evidencia de implementación; el razonamiento sobre invariantes explica por qué las actualizaciones son válidas para toda entrada que cumpla las precondiciones.

Ejemplo de fallo evitado. Si se escribiera $\ell = \text{pos}$ y la estimación repitiera el mismo índice, el intervalo podría no reducirse. Usar $\ell = \text{pos} + 1$ elimina una posición ya comprobada y garantiza progreso. Esta explicación no modifica el código entregado.

1.6 Núcleo 6. Búsqueda lineal como referencia y solución práctica

Lineal compara sucesivamente la clave con cada elemento y termina al encontrar una coincidencia. El recorrido directo encaja en el modelo de operaciones de Morin (s.f.) y en el análisis de programas de Sedgewick y Wayne (2011); Skiena (2020) aporta la perspectiva de elegir una solución según las restricciones reales. Su ventaja estructural es que no exige preparación ni orden. En el peor caso debe revisar los n elementos: cualquier posición todavía no inspeccionada podría contener la clave. Esta necesidad ofrece una justificación del costo lineal para el problema sin información adicional, no solamente una observación sobre el ciclo `for`.

El mejor caso ocurre en el primer elemento; una ausencia obliga a completar el recorrido y un hit intermedio depende de su posición. La interpretación conjunta del modelo de Morin (s.f.), las condiciones de búsqueda de Sedgewick y Wayne (2011) y los efectos de memoria de Van Sandt et al. (2019) evita descartar lineal por su notación. Para arreglos muy pequeños o pocas consultas, los gastos de preparación pueden superar el ahorro de una estrategia más elaborada. Sus accesos consecutivos favorecen localidad espacial, pero no eliminan el crecimiento de trabajo. En los tamaños evaluados, la reducción de comparaciones de binaria es suficientemente grande para prevalecer en datos ordenados; esta conclusión se limita a los resultados ya conservados.

Derivación propia. Si el acierto aparece en el índice i , lineal realiza $i+1$ igualdades; si no aparece, realiza n . Sumar esas cantidades para las consultas almacenadas explica el oráculo independiente usado en la auditoría original, sin necesitar volver a ejecutar el benchmark.

1.7 Núcleo 7. Búsqueda binaria: reducción, recurrencia y garantía

Binaria utiliza el orden para elegir un único subintervalo después de examinar su centro. Sedgewick y Wayne (2011, sección 1.1) documentan la condición de arreglo ordenado; Erickson (2019, capítulo 1) desarrolla el razonamiento recursivo y las recurrencias, y Morin (s. f., sección 1.3) proporciona el trasfondo logarítmico. La traducción propia a una recurrencia de peor caso es $T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$, salvo ajustes de extremos. Después de k reducciones quedan aproximadamente $n/2^k$ candidatos; alcanzar un intervalo de tamaño constante requiere k proporcional a $\log_2 n$. La versión iterativa implementa la misma reducción sin una pila recursiva.

La garantía depende del orden y no de la separación numérica entre claves. Esto diferencia a binaria de interpolación: grandes huecos no impiden descartar una mitad por índices. La comparación se apoya en el algoritmo de Sedgewick y Wayne (2011), la sensibilidad de interpolación de Perl et al. (1978) y el estudio práctico de Van Sandt et al. (2019). En el uniforme, el sesgado y los arreglos con repetidos se conserva la validez de binaria, aunque los duplicados alteren dónde termina. El mejor caso es encontrar la clave en el primer centro; para posiciones equiprobables y claves distintas, el promedio crece logarítmicamente. No se promete que cada búsqueda individual use exactamente $\log_2 n$ comparaciones.

Traza propia. En $[2, 4, 8, 12, 18]$, buscar 12 examina primero 8 y luego 12. Buscar 7 termina con el intervalo vacío después de descartar posiciones incompatibles. Las dos trayectorias tienen costos distintos, aunque compartan algoritmo y tamaño.

1.8 Núcleo 8. Búsqueda exponencial y límite efectivo desconocido

Exponencial separa dos problemas: localizar una cota y buscar dentro de ella. El ejercicio de lanzamiento de huevos de Sedgewick y Wayne (2011, sección 1.4) propone duplicación repetida seguida de búsqueda binaria; el análisis de reducción de Erickson (2019) y los logaritmos de Morin (s. f.) permiten justificar el esquema empleado aquí. Tras resolver el índice cero, se prueban $1, 2, 4, 8, \dots$ hasta encontrar un valor no menor que x o alcanzar el final. Para una posición objetivo $p \geq 1$, se necesitan $O(\log p)$ ampliaciones; el intervalo resultante tiene tamaño $O(p)$ y su refinamiento binario también cuesta $O(\log p)$. Una forma uniforme que contempla el inicio es $O(1 + \log(p + 1))$.

Su utilidad depende de la interfaz. Morin (s. f., secciones 1.2 y 1.4) separa operaciones disponibles y representación; Sedgewick y Wayne (2011) vinculan búsqueda y acceso a arreglos, y Skiena (2020) destaca la adecuación del diseño al problema. Una fuente sin longitud inicial puede permitir `leer(i)` con una señal segura de fin; en ese caso tiene sentido duplicar índices. Un flujo exclusivamente secuencial no ofrece ese salto a costo constante. Nuestra implementación conoce n y utiliza una cota finita: conserva el mecanismo, pero no mide la ventaja de desconocer el límite. Para claves lejanas, la fase adicional puede aportar comparaciones sin mejorar a binaria directa.

Traza propia. Si la primera posición con valor suficiente es 13, las sondas 1, 2, 4, 8 y 16 delimitan un intervalo que contiene esa posición. El refinamiento se aplica dentro de la cota, no sobre todos los índices desde cero.

1.9 Núcleo 9. Interpolación y modelo de posición por valor

Interpolación aprovecha información numérica que binaria no necesita. Estima la fracción del rango de valores ocupada por x y la proyecta sobre el rango de índices. Perl et al. (1978) estudian la búsqueda bajo uniformidad, Van Sandt et al. (2019) revisan su comportamiento en memoria y Sedgewick y Wayne (2011) ofrecen binaria como término de comparación por índices. La fórmula implementada es

$$\text{pos} = \ell + \left\lfloor \frac{(x - a[\ell])(h - \ell)}{a[h] - a[\ell]} \right\rfloor.$$

Los límites de valor deben verificarse antes de calcularla. Si los extremos son iguales, el denominador desaparece y es necesario resolver ese caso por separado. Esta estimación es un modelo de la distribución, no una posición garantizada.

El esperado $O(\log \log n)$ pertenece a un modelo probabilístico de claves uniformes; no es una garantía para toda secuencia ordenada. El vínculo con ese supuesto proviene de Perl et al. (1978), la degradación práctica se analiza en Van Sandt et al. (2019) y la distinción entre esperado y peor caso aparece en Morin (s. f., sección 1.5). En nuestra entrada consecutiva $a[i] = i$, sustituir en la fórmula da exactamente $\text{pos} = x$ para un hit dentro del rango. Esa deducción propia explica el comportamiento especialmente favorable ya observado. No demuestra que toda muestra aleatoria uniforme se resuelva en un paso ni autoriza a reemplazar la cota esperada general por $O(1)$.

Caso adverso razonado. En $[0, 1, \dots, n - 2, n^2]$ buscando $n - 2$, el valor extremo agranda mucho el denominador. Las estimaciones avanzan casi de una posición en una, de modo que puede aparecer un costo $\Theta(n)$. Este ejemplo analítico no se agregó a los datasets ni se presenta como un experimento nuevo.

1.10 Núcleo 10. Distribución, duplicados y selección de hits y misses

La distribución relevante relaciona valores con posiciones, no solo el hecho de que la secuencia esté ordenada. La hipótesis uniforme de Perl et al. (1978), el análisis de datos no uniformes de Van Sandt et al. (2019) y los modelos de entrada de Sedgewick y Wayne (2011) explican por qué debe describirse el generador. En el sesgado del proyecto, muchas posiciones corresponden a un intervalo numérico pequeño. La fracción $(x - a[\ell]) / (a[h] - a[\ell])$ puede entonces subestimar la fracción de posiciones hasta x . Binaria sigue dividiendo por índices; interpolación necesita corregir estimaciones que responden a una densidad local muy diferente de la global.

Los duplicados y la elección de consultas añaden otra capa. Morin (s. f., sección 1.2) obliga a precisar la operación solicitada, Sedgewick y Wayne (2011) advierten la dependencia de la entrada y la documentación de Python distingue muestreo y control pseudoaleatorio (Python Software Foundation, s. f.-b). Elegir hits por posición hace que los valores repetidos tengan mayor probabilidad de ser consultados: no equivale a muestrear uniformemente las claves distintas. Por otra parte, los misses originales se encuentran por encima del máximo, de modo que su descarte es especialmente sencillo para interpolación. Son propiedades del protocolo conservado; se explicitan para limitar el alcance de la interpretación, no para sustituir la carga por otra más favorable.

Lectura del sesgo. Tener exactamente 80% de valores en el rango estrecho garantiza la composición del dataset, pero no garantiza exactamente 80% de los 700 hits en ese rango. Los hits se muestrean con reemplazo y su composición particular depende de la semilla.

1.11 Núcleo 11. Rendimiento real, instrumentación y reproducibilidad

Un tiempo medido incluye el costo de ejecutar instrucciones sobre un entorno real. Sedgewick y Wayne (2011, sección 1.4) relacionan frecuencia y costo; Van Sandt et al. (2019) estudian el intercambio entre cálculo y accesos a memoria, y Google (s. f.) documenta calentamiento, repeticiones y prevención de optimizaciones en microbenchmarks. Estas perspectivas justifican separar conteo y cronómetro. En nuestro código C++ el contador se desactiva en compilación para la función medida. El contador no representa todas las operaciones: incluye comparaciones entre claves y ciertos extremos, pero excluye controles de índices, divisiones y llamadas. Por eso una menor cuenta puede orientar una explicación sin determinar por sí sola el tiempo final.

La medición de duración y su variabilidad requieren también una lectura crítica. Python Software Foundation (s. f.-c) documenta `perf_counter()` como reloj apropiado para intervalos cortos; Google (s. f.) recomienda repetición y tratamiento de estadísticas, y Morin (s. f., sección 1.5) distingue el análisis abstracto de las particularidades de ejecución. En esta práctica se conservaron tres lotes y su desviación muestral. Esa desviación describe los lotes, no un intervalo de confianza ni la variación entre todas las posibles máquinas. La semilla reproduce entradas, no estados de caché, interrupciones o carga del sistema. Las huellas SHA-256 permiten identificar archivos; tampoco demuestran por sí solas que el protocolo sea metodológicamente perfecto.

Delimitación de evidencia. La localidad espacial, las ramas y el costo de los objetos de Python son mecanismos explicativos plausibles; no se midieron fallos de caché ni errores del predictor. Los resultados originales sustentan tendencias de estas implementaciones, no una atribución causal aislada a cada componente de hardware.

1.12 Núcleo 12. Selección del algoritmo y amortización del ordenamiento

La elección debe considerar el costo completo de resolver la carga prevista. La orientación práctica de Skiena (2020), el estudio experimental de Sedgewick y Wayne (2011) y la selección según distribución de Van Sandt et al. (2019) sostienen una estrategia condicionada: lineal para pocos datos o pocas consultas sin orden; binaria como opción robusta con orden; interpolación cuando el valor predice bien la posición; exponencial cuando debe obtenerse una cota. Esto no equivale a un ranking fijo. Un muestreo puede estimar uniformidad, pero no certificar que cada pareja del arreglo esté ordenada. Si el certificado no existe, verificar todo el orden tiene un costo que debe reconocerse.

El modelo de crecimiento de Morin (s. f.), la recurrencia de ordenamiento por mezcla explicada por Erickson (2019) y la combinación de ordenar y buscar de Sedgewick y Wayne (2011) permiten plantear una comparación de costos. Como elaboración propia, si las constantes se aproximan por α , β y γ , conviene estudiar

$$T_L \approx \alpha q n, \quad T_{OB} \approx \beta n \log_2 n + \gamma q \log_2 n.$$

Ordenar y luego buscar resulta favorable en este modelo cuando $q > \beta n \log_2 n / (\alpha n - \gamma \log_2 n)$, siempre que el denominador sea positivo. No se asignan aquí valores experimentales a esas constantes. El selector simple usa una aproximación de constantes iguales, por lo que su umbral es orientativo y necesita calibración en una aplicación real.

Síntesis defendible. Los umbrales 64 y 0.05 ya propuestos no proceden de una optimización demostrada. La hipótesis de uniformidad puede fallar entre puntos muestreados; el costo de ordenar puede no amortizarse; y la necesidad de conservar índices originales puede requerir pares (valor, índice). Una recomendación responsable debe declarar estas condiciones.

Síntesis comparativa de los cuatro algoritmos

Algoritmo	Mejor	Promedio	Peor	Supuesto del promedio
Lineal	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	Posiciones equiprobables; ausencia recorre todo.
Binaria	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	Orden, claves distintas y posiciones equiprobables.
Exponencial	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	Mismo modelo; depende de la posición alcanzada.
Interpolación	$\Theta(1)$	$O(\log \log n)$	$\Theta(n)$	Esperado bajo uniformidad probabilística; no universal.

Nota. Síntesis del análisis desarrollado. Las cuatro versiones iterativas emplean $O(1)$ palabras auxiliares, excluidos entrada, consultas y estructuras del protocolo experimental. Las expectativas teóricas no son mediciones nuevas.

2. Metodología y correctitud

Se siguió la guía oficial (Zanabria Galvez, s.f.) con $n \in \{10\,000, 100\,000, 500\,000\}$. Python generó los archivos compartidos con semilla 123: (a) uniforme consecutivo $0, \dots, n-1$, sin duplicados; (b) sesgado, exactamente 80% en $[0, n/10-1]$ y 20% en $[n/10, 5n]$, con duplicados posibles, luego ordenado; (c) una permutación del uniforme. Se guardaron huellas SHA-256 de los nueve archivos.

Cada archivo contiene 1000 consultas: 700 valores tomados por posición con reemplazo y 300 ausentes $10n+i$, $0 \leq i < 300$; se mezclaron con el mismo generador. Ambos lenguajes leyeron idénticos datos y consultas. Solo lineal se ejecutó sobre desordenados. Se validaron vacío, único, repetidos, negativos, extremos, intermedios, hits, misses y 50 arreglos aleatorios; además, cada consulta experimental se comprobó antes de medir. Se acepta cualquier índice que contenga la clave; -1 solo si está ausente.

Se midieron tres lotes de 1000 búsquedas por combinación (27 combinaciones por lenguaje), con 20 consultas de calentamiento y orden de algoritmos rotado. Se usaron `steady_clock` de `std::chrono` y `time.perf_counter()`; se dividió cada lote entre 1000 y se promediaron sus tiempos. Lectura, generación, ordenamiento y validación quedaron fuera; ciclo, llamada y checksum quedaron dentro. El conteo de comparaciones se ejecutó aparte en C++, sin modificar las funciones cronometradas. Se conservaron CSV de medias, desviaciones muestrales, repeticiones y registros de consola.

3. Entorno experimental y límites del diseño

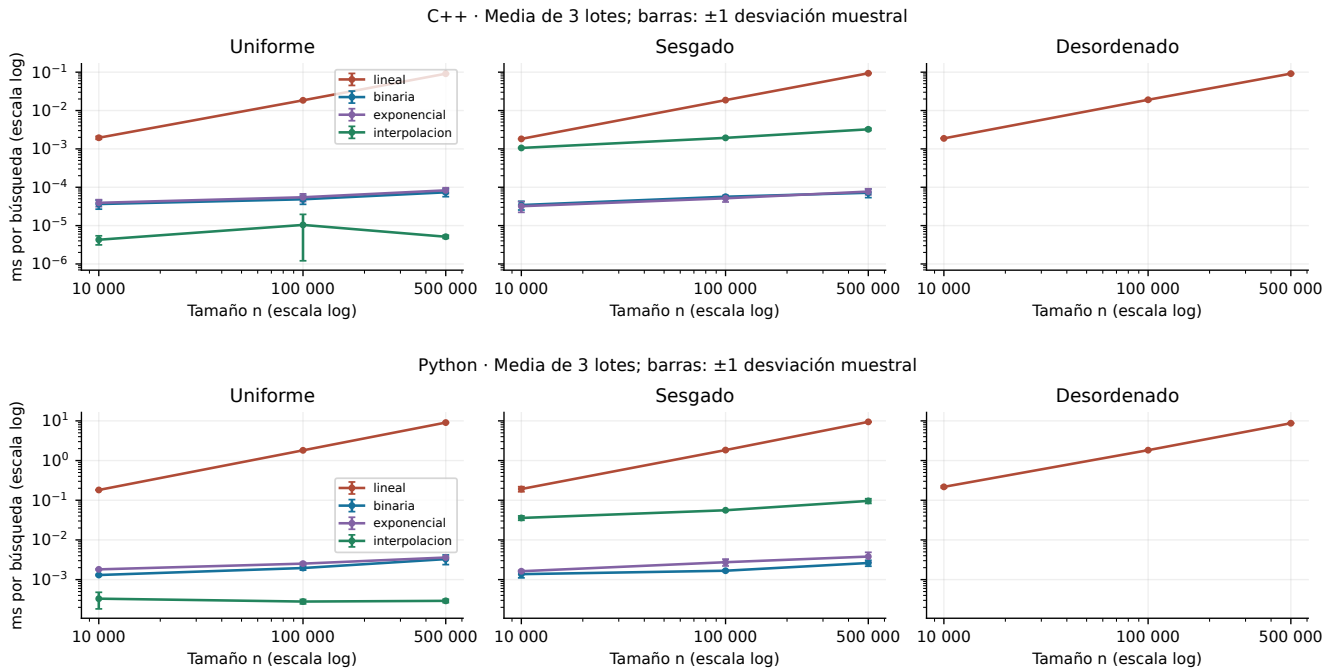
Entorno remoto de Work: Ubuntu 24.04.3 LTS, Linux-6.18.35-x86_64-with-glibc2.39; CPU visible AMD EPYC 9V74 80-Core Processor; RAM visible 16399700 KiB (aprox. 15.64 GiB), límite del contenedor 14 GiB. Python 3.12.14; g++ (Ubuntu 13.3.0-6ubuntu2 24.04) 13.3.0. Flags: `-std=c++17 -O2 -Wall -Wextra -pedantic`. Ejecución secuencial, sin fijar afinidad ni aislamiento exclusivo; el nombre del procesador no representa núcleos dedicados. Registro UTC: 2026-09-09.

Los misses externos favorecen el rechazo inmediato de interpolación. El uniforme consecutivo permite estimación exacta y no representa todos los datos aleatorios uniformes. Las pruebas previas calientan memoria; no se mide caché fría. Tres repeticiones, lotes C++ muy cortos y una sola semilla limitan inferencias: se describen tendencias, no significancia estadística ni pruebas matemáticas de complejidad.

4. Resultados experimentales

Tabla 1. Media en milisegundos por búsqueda; tres lotes, 700 hits y 300 misses por lote. Se muestran todas las combinaciones válidas; «-» indica precondition incumplida. Notación e-05 significa $\times 10^{-5}$.

Lenguaje	Dataset	n	Lineal	Binaria	Exponencial	Interpolación
C++	uniforme	10 000	0.00195063	3.65003e-05	3.92607e-05	4.27967e-06
C++	uniforme	100 000	0.018463	4.8508e-05	5.4807e-05	1.04223e-05
C++	uniforme	500 000	0.091155	7.3752e-05	8.29853e-05	5.12767e-06
C++	sesgado	10 000	0.0018231	3.43477e-05	3.20403e-05	0.00105852
C++	sesgado	100 000	0.0186869	5.6603e-05	5.15223e-05	0.00193876
C++	sesgado	500 000	0.0936415	7.18157e-05	7.71733e-05	0.00323955
C++	desordenado	10 000	0.00187829	—	—	—
C++	desordenado	100 000	0.0190711	—	—	—
C++	desordenado	500 000	0.0917346	—	—	—
Python	uniforme	10 000	0.181515	0.00130485	0.00181138	0.00033091
Python	uniforme	100 000	1.80999	0.00195744	0.00252695	0.000280049
Python	uniforme	500 000	9.05581	0.00328532	0.00360516	0.000291589
Python	sesgado	10 000	0.191669	0.00137221	0.00162627	0.0357729
Python	sesgado	100 000	1.83819	0.00167438	0.00274535	0.059274
Python	sesgado	500 000	9.44253	0.00262187	0.00380804	0.0962232
Python	desordenado	10 000	0.217395	—	—	—
Python	desordenado	100 000	1.82729	—	—	—
Python	desordenado	500 000	8.72613	—	—	—



Figuras 1 y 2. Curvas generadas directamente desde los CSV; ambos ejes son logarítmicos. Las barras representan una desviación muestral entre lotes, no intervalos de confianza. Cada lenguaje tiene su propia escala vertical.

Al multiplicar n por 50 en el uniforme, el tiempo lineal aumentó 46.7 veces en C++ y 49.9 en Python: tendencia compatible con crecimiento lineal, con constantes y ruido reales. Para $n = 500\,000$, binaria fue 1236.0 y 2756.4 veces más rápida que lineal, respectivamente. Interpolación fue 14.4 y 11.3 veces más rápida que binaria en el uniforme, pero tardó 45.1 y 36.7 veces lo de binaria en el sesgado.

Conteo independiente. En C++, para el uniforme, lineal pasó de 6542.549 a 326242.621 comparaciones; binaria de 24.878 a 35.934 e interpolación permaneció en 3.4. En el sesgado de 500 000, binaria registró 32.4 e interpolación 1600.14. Se cuentan comparaciones entre valores, incluidos límites y extremos; no controles de índices ni aritmética. El gráfico completo está en [graficos/comparaciones.pdf](#). Los conteos respaldan la interpretación sin depender de milisegundos.

5. Discusión: respuestas a las preguntas de la práctica

1. **¿Por qué binaria supera a lineal en datos ordenados?** El orden permite descartar mitades. En el uniforme grande se obtuvieron unas 36 comparaciones frente a 326 mil de lineal. La menor cantidad de pasos compensa aquí el acceso menos secuencial de binaria.
2. **¿Cuándo interpolación puede superar a binaria?** Cuando valor y posición se relacionan aproximadamente de manera lineal. En nuestros enteros consecutivos la estimación es exacta; cada hit requiere una sola posición estimada. La media de 3.4 comparaciones incluye cuatro por hit y dos por miss externo.
3. **¿Cuándo pierde la ventaja?** Ante concentraciones, huecos grandes o valores extremos que distorsionan la estimación. El sesgado lo evidencia, sin llegar a demostrar el peor caso. Un arreglo $[0, 1, \dots, n-2, n^2]$ buscando $n-2$ puede provocar avances casi unitarios y costo lineal.
4. **¿Por qué afecta el sesgo?** El 80% ocupa solo un rango numérico estrecho: la fracción de valores ya no representa la fracción de posiciones. La fórmula subestima posiciones en esa zona y necesita reiterar; binaria depende del orden, no de la uniformidad numérica.
5. **¿Para qué sirve exponencial con límite desconocido?** Duplica índices hasta superar la clave o detectar fin y luego refina con binaria. Requiere acceso indexado seguro; un flujo únicamente secuencial no permite saltos gratuitos. Aquí conocemos n : se estudia la acotación, sin simular una fuente infinita ni medir esa ventaja específica.
6. **¿Cómo influyen caché, localidad, ramas y constantes?** Lineal lee posiciones consecutivas, favorables a localidad y precarga; binaria salta y decide ramas dependientes de datos. Interpolación añade multiplicación y división. C++ usa enteros contiguos; Python maneja objetos y ejecuta bytecode. Son mecanismos plausibles (Van Sandt et al., 2019), no causas aisladas: no se midieron fallos de caché ni de predicción de ramas.
7. **¿Por qué Big O no predice milisegundos?** Omite constantes y términos menores; tampoco fija hardware, representación, distribución de consultas o carga del sistema. Exponencial y binaria comparten orden logarítmico y aun así difieren. Las pequeñas ventajas de exponencial en algunos conjuntos C++ no justifican un ganador universal, especialmente con la dispersión observada.

6. Heurística propuesta

Se propone un selector sencillo, implementado y probado aparte en `python/heuristica.py`. Sea q el número previsto de consultas y E el error máximo entre posición relativa y valor normalizado, estimado con 33 posiciones equiespaciadas:

Si límite desconocido: exponencial con acceso seguro si hay orden certificado; si no, lineal hasta fin.

Si $n < 64$: lineal. Si no hay certificado de orden, verificarlo una vez en $O(n)$.

Si desordenado: comparar qn con $n \log_2 n + q \log_2 n$; ordenar una copia y usar binaria solo si se amortiza; en otro caso, lineal.

Si extremos iguales: binaria. En otro caso, muestrear y calcular $E = \max_i \left| \frac{a[i]-a[0]}{a[n-1]-a[0]} - \frac{i}{n-1} \right|$.

Si $E \leq 0.05$: interpolación; si no, binaria.

Los umbrales 64 y 0.05 son propuestos, no calibrados. El muestreo no certifica orden ni garantiza velocidad. El costo de verificar, copiar y ordenar debe amortizarse; para una sola consulta sin metadatos puede convenir lineal directamente. El modelo supone constantes iguales y orienta una decisión, no demuestra optimalidad. Si se requieren índices originales, se preservan al ordenar pares.

7. Conclusiones

Binaria ofrece una opción estable para datos ordenados no uniformes; interpolación destacó en el caso consecutivo y perdió claramente con sesgo. Lineal evita preparación en desordenados y pocas consultas; exponencial aporta acotación cuando la interfaz admite saltos sin conocer el final. Los resultados son reproducibles desde archivos compartidos y coherentes con estas explicaciones, pero no demuestran cotas asintóticas ni se extrapolan a cualquier equipo o distribución.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Erickson, J. (2019). *Algorithms*. <https://jeffe.cs.illinois.edu/teaching/algorithms/>
- Google. (s.f.). *User guide* [Documentación de Google Benchmark]. GitHub. Recuperado el 10 de septiembre de 2026, de https://github.com/google/benchmark/blob/main/docs/user_guide.md
- Morin, P. (s.f.). *Open data structures (in Python)*. Recuperado el 10 de septiembre de 2026, de <https://opendatastructures.org/ods-python/>
- Perl, Y., Itai, A., & Avni, H. (1978). Interpolation search—a log log N search. *Communications of the ACM*, 21(7), 550–553. <https://doi.org/10.1145/359545.359557>
- Python Software Foundation. (s.f.-a). *Built-in types*. En *Python 3.12 documentation*. Recuperado el 10 de septiembre de 2026, de <https://docs.python.org/3.12/library/stdtypes.html>
- Python Software Foundation. (s.f.-b). *random—Generate pseudo-random numbers*. En *Python 3.12 documentation*. Recuperado el 10 de septiembre de 2026, de <https://docs.python.org/3.12/library/random.html>
- Python Software Foundation. (s.f.-c). *time—Time access and conversions*. En *Python 3.12 documentation*. Recuperado el 10 de septiembre de 2026, de <https://docs.python.org/3.12/library/time.html>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. <https://algs4.cs.princeton.edu/>
- Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer. <https://doi.org/10.1007/978-3-030-54256-6>
- Van Sandt, P., Chronis, Y., & Patel, J. M. (2019). Efficiently searching in-memory sorted arrays: Revenge of the interpolation search? En *Proceedings of the 2019 International Conference on Management of Data* (pp. 36–53). Association for Computing Machinery. <https://doi.org/10.1145/3299869.3300075>
- Zanabria Galvez, A. H. (s.f.). *Práctica 01 — Búsquedas y Notación Asintótica* [Guía de práctica, SIS210]. Universidad Nacional del Altiplano.